UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

Laboratorio de Fundamentos de Sistemas Embebidos

GRUPO **01**

PROFESOR: **Ing. Rubén Ulises Sandoval Rodríguez**

Practica 04

Equipo: 05

- sdsdsds
- sdsdsds
- sdsdsd
- Qintero Rubio Martin

FECHA DE ENTREGA: 26 de octubre de 2026

Practica 04 FSE

Objetivo

Al finalizar la práctica el alumno será capaz de:

- Configurar la Raspberry Pi como un punto de acceso inalámbrico.
- Conectar dispositivos móviles a la red local generada.
- Utilizar la aplicación Pytphox para exponer los datos de sensores mediante una interfaz HTTP.
- Desarrollar un programa que obtenga y grafique datos de varios teléfonos celulares.
- Controlar periféricos conectados a la Raspberry Pi mediante mediciones de sensores remotos.

Lista de materiales

- Raspberry Pi 3B+
- Laptop
- Servomotor SG90
- Buzzer activo
- Fuente de alimentación 3.3-5V USB
- Protoboard
- Cables y jumpers requeridos

Desarrollo

1. Preparación del entorno

1.1 Configuración de la RPI3b+ Cómo punto de acceso

Todas las actividades necerias para que la RPI sea un AP

Actividad 1

Tabla de direcciones

integrante	url	
Isaac	http://192.168.4.47:8080	
Martin	http:192.168.4.126:8080	

Probando acceso a un telefono

```
alumno@raspberrypi:~/04 $ curl --max-time 3 "http://192.168.4.1:8080/config"^C
alumno@raspberrypi:~/04 $ ping 192.168.4.1
PING 192.168.4.1 (192.168.4.1) 56(84) bytes of data.
64 bytes from 192.168.4.1: icmp_seq=1 ttl=64 time=0.264 ms
64 bytes from 192.168.4.1: icmp_seq=2 ttl=64 time=0.185 ms
64 bytes from 192.168.4.1: icmp_seq=3 ttl=64 time=0.216 ms
64 bytes from 192.168.4.1: icmp_seq=4 ttl=64 time=0.218 ms
64 bytes from 192.168.4.1: icmp_seq=5 ttl=64 time=0.232 ms
^C
--- 192.168.4.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4087ms
rtt min/avg/max/mdev = 0.185/0.223/0.264/0.025 ms
alumno@raspberrypi:~/04 $ curl http://192.168.4.1
curl: (7) Failed to connect to 192.168.4.1 port 80 after 0 ms: Couldn't connect to
server
alumno@raspberrypi:~/04 $ curl http://192.168.4.1
curl: (7) Failed to connect to 192.168.4.1 port 80 after 0 ms: Couldn't connect to
server
alumno@raspberrypi:~/04 $ curl http://192.168.4.1
curl: (7) Failed to connect to 192.168.4.1 port 80 after 0 ms: Couldn't connect to
server
alumno@raspberrypi:~/04 $ curl http://192.168^C.1
alumno@raspberrypi:~/04 $ ping 192.168.4.1
PING 192.168.4.1 (192.168.4.1) 56(84) bytes of data.
64 bytes from 192.168.4.1: icmp_seq=1 ttl=64 time=0.247 ms
64 bytes from 192.168.4.1: icmp_seq=2 ttl=64 time=0.161 ms
64 bytes from 192.168.4.1: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 192.168.4.1: icmp_seq=4 ttl=64 time=0.200 ms
^C
--- 192.168.4.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3059ms
rtt min/avg/max/mdev = 0.147/0.188/0.247/0.038 ms
alumno@raspberrypi:~/04 $
alumno@raspberrypi:~/04 $ ping http://192.168.4.126:8080
ping: http://192.168.4.126:8080: Name or service not known
alumno@raspberrypi:~/04 $ curl http://192.168.4.126:8080/config
{"crc32":"201e3639","title":"Acceleration (without g)","localTitle":"Aceleración (sin
g)","category":"Raw Sensors","localCategory":"Sensores","buffers":
[{"name":"acc_time","size":0},{"name":"accX","size":0},{"name":"accY","size":0},
{"name":"accZ","size":0},{"name":"acc","size":0}],"inputs":
[{"source":"linear_acceleration","outputs":[{"x":"accX"},{"y":"accY"},{"z":"accZ"},
{"abs":"acc"},{"t":"acc_time"}]}],"export":[{"set":"Raw Data","sources":
[{"label":"Time (s)","buffer":"acc_time"},{"label":"Linear Acceleration x
(m\/s^2)","buffer":"accX"},{"label":"Linear Acceleration y (m\/s^2)","buffer":"accY"},
{"label":"Linear Acceleration z (m\/s^2)","buffer":"accZ"},{"label":"Absolute
acceleration (m\/s^2)","buffer":"acc"}]}]}alumno@raspberrypi:~/04 $ curl
http://192.168.4.126:8080/config
{"crc32":"201e3639","title":"Acceleration (without g)","localTitle":"Aceleración (sin
```

```
g)","category":"Raw Sensors","localCategory":"Sensores","buffers":
[{"name":"acc_time","size":0},{"name":"accX","size":0},{"name":"accY","size":0},
{"name":"accZ","size":0},{"name":"acc","size":0}],"inputs":
[{"source":"linear_acceleration","outputs":[{"x":"accX"},{"y":"accY"},{"z":"accZ"},
{"abs":"acc"},{"t":"acc_time"}]}],"export":[{"set":"Raw Data","sources":
[{"label":"Time (s)","buffer":"acc_time"},{"label":"Linear Acceleration x
(m\/s^2)","buffer":"accX"},{"label":"Linear Acceleration y (m\/s^2)","buffer":"accY"},
{"label":"Linear Acceleration z (m\/s^2)","buffer":"accZ"},{"label":"Absolute
acceleration (m\/s^2)","buffer":"acc"}]}]}alumno@raspberrypi:~/04 $
```

Consultar tres ejes de aceleracion

```
alumno@raspberrypi:~/04 $ curl "http://192.168.4.126:8080/get?accX&accY&accZ"
{"buffer":{
"accX":{"size":0,"updateMode":"single", "buffer":[-2.084589E-2]},
"accY":{"size":0,"updateMode":"single", "buffer":[-1.5816167E-3]},
"accZ":{"size":0,"updateMode":"single", "buffer":[1.0778427E-2]}
},
"status":{
"session":"f01888", "measuring":true, "timedRun":false, "countDown":0
}
}
```

get axx x ful

```
alumno@raspberrypi:~/04 $ curl "http://192.168.4.126:8080/get?accX=full"
...
45E-2,4.445672E-3,-7.4715614E-3,-2.8586388E-4,-9.7881556E-3,2.1679401E-3,2.1468401E-
3,9.2004538E-3,4.3702126E-3,9.0817213E-3,1.8240213E-3,1.1308312E-2,8.7835789E-
3,1.5813112E-3,3.9410591E-3,-8.4543228E-4,8.6610317E-3,3.8353205E-3,-5.7017803E-
3,-8.0324411E-3,-3.2171011E-3,-3.1933784E-3,6.3327551E-3,-3.2252073E-3,-8.2540512E-
4,1.502037E-3,1.4350414E-3,6.1719418E-3,-3.3322573E-3,6.2469244E-3,1.3879538E-
3,-1.0546446E-3,3.6503077E-3,3.564477E-3,8.2831383E-3,-1.2387037E-3,-1.5537381E-
2,-1.0659575E-2,-3.4463406E-3,3.7069321E-3,3.6734343E-3]}
},
"status":{
"session":"f01888", "measuring":true, "timedRun":false, "countDown":0
}
}
alumno@raspberrypi:~/04 $
```

Programa para obtener una grafica

```python
import math
import requests
import time
import matplotlib.pyplot as plt

# 1. Registrar las URL de al menos dos teléfonos y asignar un identificador
telefonos = {
    "Teléfono 1 (126)": "http://192.168.4.126:8080",
    "Teléfono 2 (47)": "http://192.168.4.47:8080"
}

# Estructura para almacenar los datos
datos_capturados = {
    "Teléfono 1 (126)": {"tiempo": [], "aceleracion": []},
    "Teléfono 2 (47)": {"tiempo": [], "aceleracion": []}
}

duracion_total = 90 # 3. Registrar durante al menos 30 segundos
intervalo_deseado = 1 # 4. Intervalo inicial de consulta de 0,2 s
timeout_peticion = 20.0 # Aumentado debido a la alta latencia reportada (> 3s)

print("Iniciando captura de datos por 30 segundos...")
tiempo_inicio = time.monotonic()

while True:
    tiempo_actual = time.monotonic()
    tiempo_transcurrido = tiempo_actual - tiempo_inicio

    if tiempo_transcurrido >= duracion_total:
        break

    inicio_ciclo = time.monotonic()

    # Las consultas se realizan de manera secuencial
    for nombre, base_url in telefonos.items():
        try:
            # 2. Consultar periódicamente al menos un eje de aceleración (accX)
            respuesta = requests.get(base_url + "/get?accX", timeout=timeout_peticion)
            respuesta.raise_for_status()

            datos = respuesta.json()
            valores = datos["buffer"]["accX"]["buffer"]

            if not valores:
                raise ValueError("Buffer vacio")

            ultimo = valores[-1]
```

```
            if (not isinstance(ultimo, (int, float)) or not math.isfinite(ultimo)):
                raise ValueError("Medicion no valida")

            # Guardar dato exitoso
            datos_capturados[nombre]["tiempo"].append(tiempo_transcurrido)
            datos_capturados[nombre]["aceleracion"].append(ultimo)
            print(f"[{tiempo_transcurrido:.2f}s] {nombre} - Aceleración X: {ultimo}
m/s^2")

        except (requests.RequestException, ValueError, KeyError, TypeError) as error:
            print(f"[{tiempo_transcurrido:.2f}s] {nombre} - Error: {error}")
            # 5. Representar los datos faltantes como tales, sin sustituirlos por cero
            # Usar float('nan') permite que matplotlib corte la línea sin graficar un
cero
            datos_capturados[nombre]["tiempo"].append(tiempo_transcurrido)
            datos_capturados[nombre]["aceleracion"].append(float('nan'))

    fin_ciclo = time.monotonic()
    tiempo_ejecucion_ciclo = fin_ciclo - inicio_ciclo

    # Calcular cuánto tiempo dormir para mantener el intervalo de 0.2s si es posible
    tiempo_espera = intervalo_deseado - tiempo_ejecucion_ciclo
    if tiempo_espera > 0:
        time.sleep(tiempo_espera)

print("Captura finalizada. Generando gráfica...")

# 6. Generar una gráfica con las mediciones de los teléfonos
plt.figure(figsize=(10, 6))

for nombre, datos in datos_capturados.items():
    plt.plot(datos["tiempo"], datos["aceleracion"], label=nombre, marker='o',
markersize=4, linestyle='-')

# 7. Incluir tiempo en segundos y aceleración en m/s^2 en los ejes
plt.title("Aceleración en el eje X vs Tiempo")
plt.xlabel("Tiempo (s)")
plt.ylabel("Aceleración (m/s^2)")
plt.legend() # Identificar cada señal mediante una leyenda
plt.grid(True)

# Guardar el resultado
plt.savefig("comparacion.png") # Requisito del documento
print("Gráfica guardada exitosamente como 'comparacion.png'")
```

corremos el programa

```
alumno@raspberrypi:~/04/01_act $ python3 act1.py
Iniciando captura de datos por 30 segundos...
[0.00s] Teléfono 1 (126) - Aceleración X: 2.0142529 m/s^2
[0.00s] Teléfono 2 (47) - Aceleración X: -0.008015871 m/s^2
[20.74s] Teléfono 1 (126) - Aceleración X: -1.0206672 m/s^2
[20.74s] Teléfono 2 (47) - Aceleración X: 0.014784127 m/s^2
[30.09s] Teléfono 1 (126) - Aceleración X: -0.71795005 m/s^2
[30.09s] Teléfono 2 (47) - Aceleración X: 0.0067940354 m/s^2
[42.99s] Teléfono 1 (126) - Aceleración X: 0.49629354 m/s^2
[42.99s] Teléfono 2 (47) - Aceleración X: -0.0099852681 m/s^2
[43.99s] Teléfono 1 (126) - Aceleración X: 2.0346196 m/s^2
[43.99s] Teléfono 2 (47) - Aceleración X: 0.019134045 m/s^2
[52.20s] Teléfono 1 (126) - Aceleración X: 0.97627503 m/s^2
[52.20s] Teléfono 2 (47) - Aceleración X: 0.0028701723 m/s^2
[53.20s] Teléfono 1 (126) - Error: HTTPConnectionPool(host='192.168.4.126',
port=8080): Max retries exceeded with url: /get?accX (Caused by
ConnectTimeoutError(<urllib3.connection.HTTPConnection object at 0x7f7b8cf090>,
'Connection to 192.168.4.126 timed out. (connect timeout=20.0)'))
[53.20s] Teléfono 2 (47) - Aceleración X: 0.0040756166 m/s^2
[74.30s] Teléfono 1 (126) - Aceleración X: 1.8465383 m/s^2
[74.30s] Teléfono 2 (47) - Aceleración X: 0.019976586 m/s^2
[75.30s] Teléfono 1 (126) - Aceleración X: -0.62597179 m/s^2
[75.30s] Teléfono 2 (47) - Error: HTTPConnectionPool(host='192.168.4.47', port=8080):
Max retries exceeded with url: /get?accX (Caused by
ConnectTimeoutError(<urllib3.connection.HTTPConnection object at 0x7f7b8d9ad0>,
'Connection to 192.168.4.47 timed out. (connect timeout=20.0)'))
Captura finalizada. Generando gráfica...
Gráfica guardada exitosamente como 'comparacion.png'
alumno@raspberrypi:~/04/01_act $
```

obtenemos como resultado esta gráfica

Actividad 2

Consiste en controlar el servomotor y el buzer mediante los movimiento de dos teléfonos conectados por WIFI

Conclusiónes

- sdsd
- sdsds
- sdsd
- **Quintero Rubio Martin**: Fue maravilloso